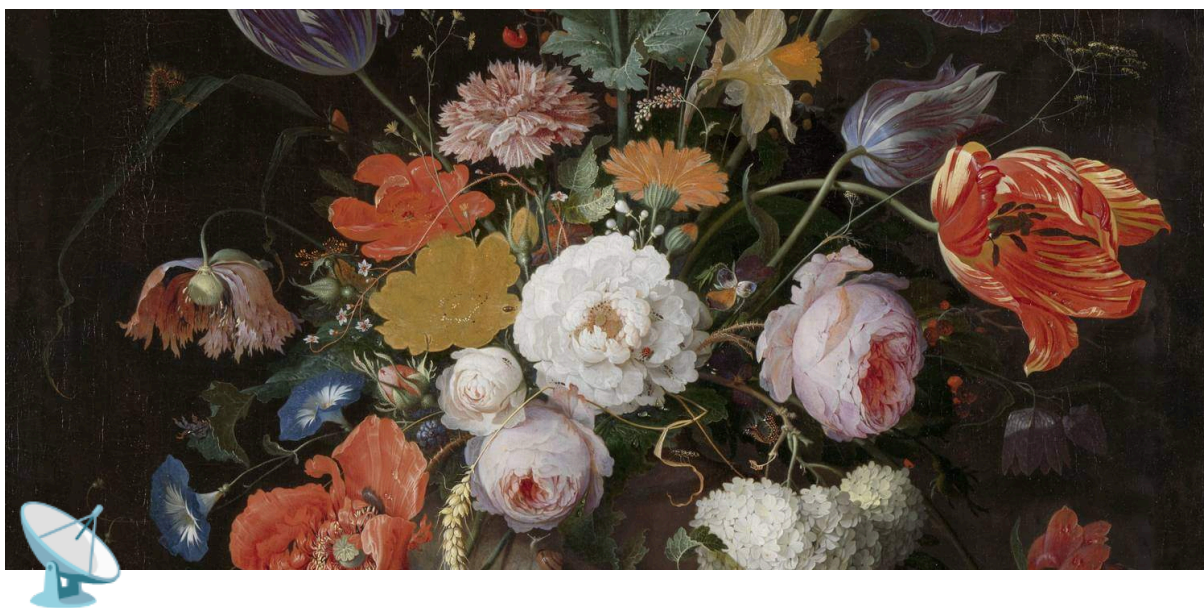




SQL

⦿ type	Post
⦿ status	Published
📅 date	@2026/01/05
≡ slug	sql
≡ summary	代码段为重点
≡ tags	实用教程 工具 必看精选
⦿ category	知识小记
≡ 标签	
✧ 状态	进行中

语法速查

1. 基本数据结构

表 — 结构化的文件，存储特定类型的数据。如：清单、目录等。

1. 每个表中数据的类型都相同。
2. 每个表都有独属于自己的唯一名字来表示自身。
3. 表的一些特性定义了数据再表中的存储方式，包含怎样的数据，数据如何分解等信息。

列



表中的一个字段，所有表都是由一个或多个竖向的列组成的。

每个表列都有相应的数据类型，它限制（或允许）该列中存储的数据。

行



表中的一个记录

表中的数据是按行存储的，所保存的每个记录存储在自己的行内。如果将表想象为网格，网格中垂直的列为表列，水平行为表行。

主键



一列（或一组列），其值能够唯一标识表中每一行。

表中的每一（几）行/列都可以唯一标识自己。而唯一标识表中每行的**这个列称为 主键**。用它来表示一个特定的行。没有主键，更新或者删除表中特定行极为困难。

列可充当主键的条件：

- ❑ 任意两行都不具有相同的主键值；
- ❑ 每一行都必须具有一个主键值（主键列不允许 NULL 值）；
- ❑ 主键列中的值不允许修改或更新；
- ❑ 主键值不能重用（如果某行从表中删除，它的主键不能赋给以后的新行）。

2. 数据检索 - SELECT

必须指明选择的对象以及从何处选择。

基本检索

```
SELECT name1, name2, FROM products;
```

对象1 对象2 对象范围

检索所有列

```
SELECT * FROM Products
```



一般而言，尽量不使用 * 通配符。因为检索不需要的列会降低检索和应用程序的性能。

检索不同值 — 使用DISTINCT关键字。

```
SELECT DISTINCT id FROM Products;
```

尽管在id所标识的列中会有多个相同值的行，现在只会输出不同id的行。

限制结果

当我们想只返回某一数量的行时，SQL并不支'0'。

MySQL

```
SELECT prod_name  
FROM Products  
LIMIT 5;
```

在SQL server 和 ACCESS 中：

```
-- 只返回前五  
SELECT TOP 5 prod_name  
FROM Products;
```

DB2

```
SELECT prod_name  
FROM Products
```

```
FETCH FIRST 5 ROWS ONLY;
```

Oracle

```
SELECT prod_name  
FROM Products  
WHERE ROWNUM <=5
```

注释方法

1. `--` 这是一条注释
2. `#` 这是一条注释
3. `/*`这是
一段注释`*/`

3. 排序检索数据 - ORDER BY

排序



子句SQL语句由子句构成，有些子句可选有些必须。一个子句通常由一个关键字加上所提供的数据组成。如上面提到的FROM就是 SELECT 的子句。在指定一条 ORDER BY 子句时，应该保证它是 SELECT 语句中最后一条子句。如果它不是最后的子句，将会出现错误消息。

```
SELECT prod_name  
FROM Products  
ORDER BY prod_name; -- 默认以prod_name 的字母顺序对数据排列
```

按多个列排序

简单指定列名，在ORDER BY 后的两个列名用逗号分开即可。

```
SELECT prod_id, prod_price, prod_name
FROM Products
ORDER BY prod_price, prod_name;
```

-- 输出

```
prod_id prod_price  prod_name
-----
BNBG02  3.4900      Bird bean bag toy
BNBG01  3.4900      Fish bean bag toy
BNBG03  3.4900      Rabbit bean bag toy
RGAN01  4.9900      Raggedy Ann
BR03    11.9900 18  inch teddy bear
```

按列位置排序

ORDER BY 还支持按相对列位置进行排序。

若我们希望先按照SELECT清单中的第二个列排序然后按照第三个列排序：

```
SELECT prod_id, prod_price, prod_name
FROM Products
ORDER BY 2, 3;
```

指定排序方向

降序： DESC

升序： ASC

```
SELECT prod_id, prod_price, prod_name
FROM Products
ORDER BY prod_price DESC, prod_name;
-- 按prod_price降序排序，并在其值相同时按照prod_name的字母顺序排序
```

4. 过滤 - WHERE



通常只会根据特定操作或报告的需要提取表数据的子集。只检索所需数据需要指定搜索条件（search criteria），搜索条件也称为过滤条件（filter condition）。
数据根据 WHERE 子句中指定的搜索条件进行过滤。

```
-- 从 products 表中检索两个列，但不返回所有行，只返回prod_price 值为 3.49 的行
SELECT prod_name, pro_price
FROM Products
WHERE pro_price = 3.49;
```



在同时使用 ORDER BY 和 WHERE 子句时，应该让 ORDER BY 位于 WHERE 之后，否则将会产生错误

WHERE 子句操作符

表 4-1 WHERE 子句操作符

操作符	说明
=	等于
<>	不等于
!=	不等于
<	小于
<=	小于等于
!<	不小于
>	大于
>=	大于等于
!>	不大于
BETWEEN	在指定的两个值之间
IS NULL	为 NULL 值



!=和<>通常可以互换。但是，并非所有 DBMS 都支持这两种不等于操作符。

```
-- 检索所有价格小于等于 10 美元的产品
SELECT prod_name, prod_price
FROM Products
WHERE prod_price <= 10;
```

不匹配检查 - != 和 <>

```
-- 列出所有不是供应商 DLL01 制造的产品
SELECT vend_id, prod_name
FROM Products
WHERE vend_id <> 'DLL01';
```



单引号用来限定字符串。如果将值与字符串类型的列进行比较，就需要限定引号。

用来与数值列进行比较的值不用引号。

范围值检查 - BETWEEN

它需要两个值，即范围的开始值和结束值。

```
-- 检索价格在 5 美元和 10 美元之间的所有产品。
SELECT prod_name, prod_price
FROM Products
WHERE prod_price BETWEEN 5 AND 10;
```

空值检查 - IS NULL



NULL

无值 (no value)，它与字段包含 0、空字符串或仅仅包含空格不同。

```
-- 返回所有没有价格（空 prod_price 字段，不是价格为 0）的产品
SELECT prod_name
FROM Products
WHERE prod_price IS NULL;
```

组合 WHERE 子句

AND 操作符 – 给 WHERE 子句附加条件

```
-- 检索由供应商 DLL01 制造且价格小于等于 4 美元的所有产品的名称和价格
SELECT prod_id, prod_price, prod_name
FROM Products
WHERE ven_id = 'DLL01' AND prod_price <= 4;
```

OR 操作符 – 检索匹配任一条件的行



第一个条件得到满足的情况下，就不再计算第二个条件

```
-- 检索由任一个指定供应商制造的所有产品的产品名和价格
SELECT prod_name, prod_price
FROM Products
WHERE vend_id = 'DLL01' OR vend_id = 'BRS01';
```

求值顺序



SQL在处理 OR 操作符前，优先处理 AND 操作符

```
-- 错误写法
-- 返回的行中有 4 行价格小于 10
美元，但是我们需要列出价格为 10美
元及以上。
```

```
-- 用括号提高 OR 的优先级
SELECT prod_name, prod_pric
e
FROM Products
```



```

SELECT prod_name, prod_price
FROM Products
WHERE vend_id = 'DLL01' OR
vend_id = 'BRS01'
AND prod_price >= 10;
--输出
prod_name prod_price
-----
---
Fish bean bag toy 3.4900
Bird bean bag toy 3.4900
Rabbit bean bag toy 3.4900
18 inch teddy bear 11.9900
Raggedy Ann 4.9900

```

```

WHERE (vend_id = 'DLL01' OR
vend_id = 'BRS01')
AND prod_price >= 10;

```

IN 操作符号 - 指定条件范围



范围中的每个条件都可以进行匹配。

其实IN 和 OR 具有相同的功能,但是IN的最大优点在于可以包含其他SELECT语句,能动态地建立WHERE子句。

```

-- 检索由供应商 DLL01 和 BRS01
制造的所有产品
SELECT prod_name, prod_price
FROM Products
WHERE vend_id IN ( 'DLL01',
' BRS01' )
ORDER BY prod_name;

```

```

-- same
SELECT prod_name, prod_price
FROM Products
WHERE vend_id = 'DLL01' OR
vend_id = 'BRS01'
ORDER BY prod_name;

```

NOT 操作符 - 否定其后所跟的任何条件。

```

SELECT prod_name
FROM Products

```

```
WHERE NOT ven_id = 'DLL01'  
ORDER BY prod_name;
```

5 用通配符进行过滤



1、通配符 (wildcard)

用来匹配值的一部分的特殊字符。为在搜索子句中使用通配符，必须使用 LIKE 操作符。但是通配符只能用于文本字段（字符串），非文本数据类型字段不能使用通配符搜索。

2、搜索模式 (search pattern)

由面值、通配符或两者构成的搜索条件。

谓词 - 当操作符作为谓词时就不再充当操作符

通配符 %(in DBMS) * (in MA)



%表示任何字符出现任意次数。

```
-- 匹配任何位置上包含文本 bean bag  
的值  
SELECT prod_id, prod_name  
FROM Products  
WHERE prod_name LIKE '%bean bag%';
```

```
-- 找出所有以词 Fish 起头的  
产品  
SELECT prod_id, prod_name  
FROM Products  
WHERE prod_name LIKE 'FISH%';
```



% 不能与NULL匹配

通配符 下划线 (_) 单个 _ 只匹配单个字符



1. DB2 不支持 _
2. Access 中为 ?

```
-- 搜索模式给出了后面跟有文本的两个通配符
SELECT prod_id, prod_name
FROM Products
WHERE prod_name LIKE '__ inch teddy bear';
```

通配符 ([]) - 必须匹配指定位置（通配符的位置）的一个字符

```
-- 找出所有名字以 J 或 M 起头的联系人
-- [JM] 匹配方括号中任意一个字符，它也只能匹配单个字符
-- % 通配符匹配第一个字符之后的任意数目的字符，返回所需结果
SELECT cust_contact
FROM Customs
WHERE cust_contact LIKE '[JM]%'
ORDER BY cust_contact;
```



如果使用的是 Microsoft Access，需要用 ! 而不是 ^ 来否定一个集合。

6. 计算字段



运行时在 SELECT 语句内创建的。

拼接字段



1. 存储在vend_name列中的名字；
2. 包含一个空格和一个左圆括号的字符串；
3. 存储在vend_country列中的国家；
4. 包括一个右圆括号的字符串。

```
-- || 与 + 所表示的含义相同
SELECT vend_name + '(' + vend_country + ')'
-- SELECT vend_name || ' ('
|| vend_country || ')'
FROM Vendors
ORDER BY vend_name;
```

Mysql 或者 MariaDB 中实现相同方法：

```
-- contact (ob1, ob2)
SELECT Concat(vend_name, '(', vend_country, ')')
FROM Vendors
ORDER BY vend_name;
```



RTRIM(): 去掉字符串右边的空格
LTRIM() (去掉字符串左边的空格)
TRIM() (去掉字符串左右两边的空格)

```
SELECT RTRIM(vend_name) + '(' + RTRIM(vend_country) + ')'
FROM Vendors
ORDER BY vend_name;
```

别名



SELECT 语句可以很好地拼接地址字段，但是拼接好的字段没有名字，只是一个值。

别名 (alias) 是一个字段或值的替换名。用 AS 关键字赋予。

别名的名字既可以是一个单词，也可以是一个字符串。如果是后者，字符串应该括在引号中。

<pre>SELECT RTRIM (vend_namej) + '(' + RTRIM)vend_country) + ')' AS vend_title FROM vendors ORDER BY vend_name;</pre>	<pre>vend_title ----- ---- Bear Emporium (USA) Bears R Us (USA) Doll House Inc. (USA) Fun and Games (England) Furball Inc. (USA) Jouets et ours (France)</pre>
--	--

算数运算

```
-- 检索订单号 20008 中的所有物品  
select prod_id, quantity, item_price  
from OrderItems  
where order_num = 20008;
```

```
-- 汇总总物品的价格 (单价*订购数量)  
SELECT prod_id,  
       quantity,  
       item_price,  
       quantity*item_price AS expanded_price  
FROM OrderItems  
WHERE order_num = 20008;
```

7. 函数



不同的DBMS所支持的函数往往不尽相同。所以在使用函数后尽量做好注释
^^

使用函数

1. 文本处理
2. 算术操作
3. 处理日期和时间值并提取特定数据
4. 返回DBMS正在使用的特殊信息

文本处理函数

1. upper(): 将文本转换为大写

```
SELECT vend_name, UPPER(vend_name) AS vend_name_upcase
FROM Vendors
ORDER BY vend_name
```

2. SOUNDEX(): 是一个将任何文本串转换为描述其语音表示的字母数字模式的算法。

```
-- 使用 SOUNDEX()函数进行搜索，它匹配所有发音类似于Michael Gre
en 的联系名：
SELECT cust_name, cust_contact
FROM Customers
WHERE SOUNDEX(cust_contact) = SOUNDEX('Michael Green');
```

表7-1 常用的文本处理函数

函数	说明
LEFT() (或使用子字符串函数)	返回字符串左边的字符
RIGHT() (或使用子字符串函数)	返回字符串右边的字符
LENGTH() (也使用 DATALENGTH() 或 LEN())	返回字符串的长度
LOWER() (Access 使用 LCASE())	将字符串转换为小写
UPPER() (Access 使用 UCASE())	将字符串转换为大写
LTRIM()	去掉字符串左边的空格

函数	说明
<code>RTRIM()</code>	去掉字符串右边的空格
<code>SOUNDEX()</code>	返回字符串的 SOUNDEX 值
<code>UPPER()</code> (Access使用UCASE())	将字符串转换为大写

时间和日期处理函数

```
-- 检索 2012 年的所有订单
1. SQL server
SELECT order_num
FROM Orders
WHERE DATEPART(yy, order_date) = 2012;

2. Access
SELECT order_num
FROM Orders
WHERE DATEPART('yyyy', order_date) = 2012;
```

数值处理函数

表7-2 常用数值处理函数

函数	说明
<code>ABS()</code>	返回一个数的绝对值
<code>COS()</code>	返回一个角度的余弦
<code>EXP()</code>	返回一个数的指数值
<code>PI()</code>	返回圆周率
<code>SIN()</code>	返回一个角度的正弦
<code>SQRT()</code>	返回一个数的平方根
<code>TAN()</code>	返回一个角度的正切

8. 汇总

聚集函数



对某些行运行的函数，计算并返回一个值。

表8-1 SQL聚集函数

函数	说明	例子	用法
<code>AVG()</code>	返回某列的平均值	<code>AVG(prod_price) AS avg_price</code>	返回 Products 表中所有产品的平均价格
<code>COUNT()</code>	返回某列的行数	<code>COUNT(*) AS num_cust</code>	返回 Customers 表中顾客的总数
<code>MAX()</code>	返回某列的最大值	<code>MAX(prod_price) AS max_price</code>	返回 Products 表中最贵物品的价格
<code>MIN()</code>	返回某列的最小值	<code>MIN(prod_price) AS min_price</code>	返回 Products 表中最便宜物品的价格
<code>SUM()</code>	返回某列值之和	<code>SUM(quantity) AS items_ordered</code>	返回订单中所有物品数量之和

聚集不同值

1. 对所有行执行运算，指定ALL参数或者不指定参数
2. 只包含不同的值，指定DISTINCT参数

```
SELECT AVG(DISTINCT prod_price) AS avg_price
FROM Products
WHERE vend_id = 'DLL01';
```



在使用了 DISTINCT 后，此例子中的 avg_price 比较高，因为有多多个物品具有相同的较低价格。排除它们提升了平均价格。



DISTINCT 不能用于 COUNT(*)

组合聚集函数

```
SELECT COUNT(*) AS num_items, price_min, price_max, price_avg
```



```

MIN(prod_price) AS price      -----
_min,
MAX(prod_price) AS price      9 3.4900 11.9900 6.823333
_max,
AVG(prod_price) AS price
_avg
FROM Products;

```

9. 分组数据

创建分组 - SELECT 语句的 GROUP BY 子句

```

-- 指定了两个列：vend_id 包含产品供应商的 ID，num_prods 为计算字段
   (用 COUNT(*)函数建立)。
-- GROUP BY 子句指示DBMS 按 vend_id 排序并分组数据。这就会对每个
   vend_id 而不是整个表计算 num_prods 一次。
SELECT vend_id, COUNT(*) AS num_prods
FROM Products
GROUP BY vend_id;

```



1. GROUP BY 子句可以包含任意数目的列，因而可以对分组进行嵌套，更细致地进行数据分组。
2. 如果在 GROUP BY 子句中嵌套了分组，数据将在最后指定的分组上进行汇总。
3. GROUP BY 子句中列出的每一列都必须是检索列或有效的表达式（但不能是聚集函数）。如果在 SELECT 中使用表达式，则必须在 GROUP BY 子句中指定相同的表达式。不能使用别名。
4. 大多数 SQL 实现不允许 GROUP BY 列带有长度可变的数据类型（如文本或备注型字段）。
5. 除聚集计算语句外，SELECT 语句中的每一列都必须在 GROUP BY 子句中给出。
6. 如果分组列中包含具有 NULL 值的行，则 NULL 将作为一个分组返回。
如果列中有多行 NULL 值，它们将分为一组。
7. GROUP BY 子句必须出现在 WHERE 子句之后，ORDER BY 子句之前

过滤分组 - HAVING 子句



HAVING 非常类似于 WHERE。事实上，目前为止所学过的所有类型的 WHERE 子句都可以用 HAVING 来替代。唯一的差别是，WHERE 过滤行，而 HAVING 过滤分组。

WHERE 在数据分组前进行过滤，HAVING 在数据分组后进行过滤。

使用 HAVING 时应该结合 GROUP BY 子句，而 WHERE 子句用于标准的行级过滤。

```
SELECT cust_id, COUNT(*) AS  
orders  
FROM Orders  
GROUP BY cust_id  
HAVING COUNT(*) >= 2;
```

```
cust_id orders  
-----  
1000000001 2
```

分组和排序

表9-1 ORDER BY 与 GROUP BY

特性	ORDER BY	GROUP BY
作用	对产生的输出排序	对行分组，但输出可能不是分组的顺序
可使用的列	任意列都可以使用（甚至非选择的列也可以使用）	只可能使用选择列或表达式列，而且必须使用每个选择列/表达式
是否必须	不一定需要	如果与聚集函数一起使用列（或表达式），则必须使用



不要忘记 ORDER BY

一般在使用 GROUP BY 子句时，应该也给出 ORDER BY 子句。这是保证数据正确排序的唯一方法。千万不要仅依赖 GROUP BY 排序数据。

```
-- 按订购物品的数目排序输出
SELECT order_num, COUNT(*) AS items
FROM OrderItems
GROUP BY order_num
HAVING COUNT(*) >= 3
ORDER BY items, order_num;
```

SELECT 子句顺序

表9-2 SELECT子句及其顺序

子句	说明	是否必须使用
SELECT	要返回的列或表达式	是
FROM	从中检索数据的表	仅在从表选择数据时使用
WHERE	行级过滤	否
GROUP BY	分组说明	仅在按组计算聚集时使用
HAVING	组级过滤	否
ORDER BY	输出排序顺序	否

10. 子查询

子查询进行过滤

若现在我们希望：

1. 检索包含物品 RGAN01 的所有订单的编号。
2. 检索具有前一步骤列出的订单编号的所有顾客的 ID。
3. 检索前一步骤返回的所有顾客 ID 的顾客信息。

```
-- DBMS 实际上必须执行三条 SELECT 语句。最里边的子查询返回订单号列表，此列表用于其外面的子查询的 WHERE 子句。  
-- 外面的子查询返回顾客 ID 列表，此顾客 ID 列表用于最外层查询的 WHERE 子句。最外层查询返回所需的数据。  
SELECT Customers  
WHERE cust_id IN (SELECT cust_id  
                  FROM Orders  
                  WHERE order_num IN (SELECT order_num  
                                       FROM OrderItems  
                                       WHERE prod_id = 'RGAN01'));
```

子查询作为计算字段

如需要显示 Customers 表中每个顾客的订单总数。订单与相应的顾客 ID 存储在 Orders 表中：

1. 从 Customers 表中检索顾客列表；
2. 对于检索出的每个顾客，统计其在 Orders 表中的订单数目。

```
-- 要对每个顾客执行 COUNT(*)，应该将它作为一个子查询  
-- 对 Customers 表中每个顾客返回三列：cust_name、cust_state 和 orders。  
-- orders 是一个计算字段，它是由圆括号中的子查询建立的。该子查询对检索出的每个顾客执行一次。  
SELECT cust_name,  
       cust_state,  
       (SELECT COUNT(*)  
        FROM Orders
```

```
cust_name cust_state  
orders  
-----  
-----  
---  
Fun4All IN 1  
Fun4All AZ 1  
Kids Place OH 0  
The Toy Store IL 1  
Village Toys MI 2
```

```
WHERE Orders.cust_id = Customers.cust_id) AS orders
FROM Customers
ORDER BY cust_name;
```

11. 联结表

联结

关系表 - 把信息分解成多个表，一类数据一个表。各表通过某些共同的值互相关联



关系数据可以有效地存储，方便地处理。因此，关系数据库的可伸缩性（能够适应不断增加的工作量而不失败。）远比非关系数据库要好。

主键（PRIMARY）：用于唯一标识每一行记录的一列或若干列的集合。每个表只能有一个主键，主键列的值必须是唯一且不能为 NULL。

建立联结 - 指定要联结的所有表以及关联它们的方式即可

```
-- 所指定的两列（prod_name 和 prod_price）在一个表中，而另一列（vend_name）在另一个表中。
-- FROM子句列出了两个表：Vendors 和 Products。它们就是这条 SELECT 语句联结的两个表的名字。
-- WHERE 子句指示 DBMS 将 Vendors 表中的 vend_id 与 Products 表中的 vend_id 匹配。
```

```
SELECT vend_name, prod_name, prod_price
FROM Vendors, Products
WHERE Vendors.vend_id = Products.vend_id;
```



在引用的列可能出现歧义时，必须使用完全限定列名（用一个句点分隔表名和列名）。如果引用一个没有用表名限制的具有歧义的列名，大多数 DBMS 会返回错误。

如上例中，要匹配的两列指定为 `Vendors.vend_id` 和 `Products.vend_id`。这里需要这种完全限定列名，如果只给出 `vend_id`，DBMS 就不知道指的是哪一个（每个表中有一个）。从前面的输出可以看到，一条 `SELECT` 语句返回了两个不同表中的数据。

WHERE 子句

在联结两个表时，实际要做的是将第一个表中的每一行与第二个表中的每一行配对。`WHERE` 子句作为过滤条件，只包含那些匹配给定条件（这里是联结条件）的行。

没有 `WHERE` 子句，第一个表中的每一行将与第二个表中的每一行配对，而不管它们逻辑上是否能配在一起。

-- 这里返回的数据用每个供应商匹配了每个产品，包括了供应商不正确的产品（即使供应商根本就没有产品）。

```
SELECT vend_name, prod_name, prod_price
FROM Vendors, Products;
```



要保证所有联结都有 `WHERE` 子句，否则 DBMS 将返回比想要的数据多得多的数据。

笛卡尔积：由没有联结条件的表关系返回的结果为笛卡儿积。检索出的行的数目将是第一个表中的行数乘以第二个表中的行数。

有时，返回笛卡儿积的联结，也称叉联结（`cross join`）。

内联结 - 等值联结（`equijoin`），基于两个表之间的相等测试。

- 前面的 `SELECT` 语句相同，但 `FROM` 子句不同
- 两个表之间的关系是以 `INNER JOIN` 指定的部分 `FROM` 子句。
- 在使用这种语法时，联结条件用特定的 `ON` 子句而不是 `WHERE` 子句给出。

传递给 `ON` 的实际条件与传递给 `WHERE` 的相同。

```
SELECT vend_name, prod_name, prod_price
```

```
FROM Vendors INNER JOIN Products
ON Vendors.vend_id = Products.vend_id;
```

联结多个表

```
-- 显示订单 20007 中的物品。订单物品存储在 OrderItems 表中。
-- 每个产品按其产品 ID 存储，它引用 Products 表中的产品。
-- 这些产品通过供应商 ID 联结到 Vendors 表中相应的供应商，供应商 ID
   存储在每个
   产品的记录中。
SELECT prod_name, vend_name, prod_price, quantity
FROM OrderItems, Products, Vendors
WHERE Products.vend_id = Vendors.vend_id
      AND OrderItems.prod_id = Products.prod_id
      AND order_num = 20007;
```

下面是用 子查询 和 联结 方式返回订购产品RGAN01 的顾客列表：

```
-- 子查询
SELECT cust_name, cust_contact
FROM Customers
WHERE cust_id IN (SELECT cust_id
                  FROM Orders
                  WHERE order_num IN (SELECT order_num
                                      FROM OrderItems
                                      WHERE prod_id = 'RGAN01'));
```

```
-- 联结
SELECT cust_name, cust_contact
FROM Customers, Orders, OrderItems
WHERE Customers.cust_id = Orders.cust_id
      AND OrderItems.order_num = Orders.order_num
      AND prod_id = 'RGAN01';
```

12. 高级联结

表别名

```
-- 给某列起别名
select rtrim(vend_name) + ' ('+ rtrim(vend_ccountry)+ ' )'
       as vend_title
from vendors
order by vend_name;
```

```
-- 表别名
-- Customers AS C 使用 C 作为 Customers 的别名，
select cust_name, cust_contact
from customers as c, order as o, orderitem as oi
where c.cust_id = o.cust_id
      and oi.order_num = o.cust_num
      and prod_id = 'RGAN01';
```



在 oracle 中应该是 Customers C, 而不是 Customers AS C

不同类型的联结

自联结

```
-- 使用子查询
-- 首先找出 Jim Jones 工作的公
司，然后找出在该公司工作的顾客。
SELECT cust_id, cust_name,
       cust_contact
FROM Customers
WHERE cust_name = (SELECT c
                    ust_name
                    FROM Customers
                    WHERE cust_contact = 'J
                    im Jones');
```

```
-- 使用联结的相同查询
SELECT c1.cust_id, c1.cust_
name, c1.cust_contact
FROM Customers AS c1, Custo
mers AS c2
WHERE c1.cust_name = c2.cus
t_name
      AND c2.cust_contact =
'Jim Jones';
```

此查询中需要的两个表实际上是相同的表，因此 Customers 表在 FROM子句中出现了两次。

自然联结

自然联结要求用户只能选择那些唯一的列，一般通过对一个表使用通配符（SELECT *），而对其他表的列使用明确的子集来完成。

-- 在这个例子中，通配符只对第一个表使用。所有其他列明确列出，所以没有重复的列被检索出来。

```
select c.*, o.order_num, o.order_date,  
       oi.prod_id, oi.quantity, oi.item_price  
from customers as c, orderr as o, orderitem as oi  
WHERE C.cust_id = O.cust_id  
      AND OI.order_num = O.order_num  
      AND prod_id = 'RGAN01';
```

外联结

联结包含了那些在相关表中没有关联行的行。

-- 检索包括没有订单顾客在内的所有顾客
-- 用关键字 OUTER JOIN来指定联结类型（而不是在 WHERE 子句中指定）。
-- 使用 LEFT OUTER JOIN 从 FROM 子句左边的表（Customers 表）中选择所有行。

```
SELECT Customers.cust_id, Orders.order_num  
FROM Customers LEFT OUTER JOIN Orders  
ON Customers.cust_id = Orders.cust_id;
```

在使用 OUTER JOIN 语法时，必须使用 RIGHT 或 LEFT 关键字指定包括其所有行的表

（RIGHT 指出的是 OUTER JOIN 右边的表，而 LEFT 指出的是 OUTER JOIN 左边的表）。



还存在另一种外联结，就是全外联结（full outer join），它检索两个表中的所有行并关联那些可以关联的行。与左外联结或右外联结包含一个表的不关联的行不同，全外联结包含两个表的不关联的行。

```
SELECT Customers.cust_id, Orders.order_num  
FROM Orders FULL OUTER JOIN Customers
```

```
ON Orders.cust_id = Customers.cust_id;
```

使用带聚集函数的联结

-- 检索所有顾客及每个顾客所下的订单数，下面的代码使用 COUNT()函数完成此工作：

```
SELECT Customers.cust_id,  
       COUNT(Orders.order_num) AS num_ord  
FROM Customers INNER JOIN Orders --对 Customers 与 Orders 做  
内连接 (INNER JOIN)，只保留在 Orders 中有对应订单的客户。  
       ON Customers.cust_id = Orders.cust_id  
GROUP BY Customers.cust_id;
```

-- 使用左外部联结来包含所有顾客。

```
SELECT Customers.cust_id,  
       COUNT(Orders.order_num) AS num_ord  
FROM Customers LEFT OUTER JOIN Orders  
       ON Customers.cust_id = Orders.cust_id  
GROUP BY Customers.cust_id;
```



使用联结和联结条件：

1. 在一个联结中可以包含多个表，甚至可以对每个联结采用不同的联结类型。虽然这样做是合法的，一般也很有用，但应该在一起测试它们前分别测试每个联结。
2. 应该总是提供联结条件，否则会得出笛卡儿积

13. 组合查询

允许执行多个查询（多条 SELECT 语句），并将结果作为一个查询结果集返回。

1. 在一个查询中从不同的表返回数据结构。
2. 对一个表执行多个查询，按一个查询返回数据。



组合查询和多个 WHERE 条件

任何具有多个WHERE 子句的 SELECT 语句都可以作为一个组合查询

创建组合查询 - union

使用 union



给出每条 SELECT 语句，在各条语句之间放上关键字 UNION。

-- 需要 Illinois、Indiana 和 Michigan 等美国几个州的所有顾客的报表，还想包括不管位于哪个州的所有 Fun4All。

```
select cust_name, cust_contact, cust_email -- 检索出这些州的所有行
from customers
where cust_state in ('IL','IN','MI');
```

SELECT cust_name, cust_contact, cust_email -- 利用简单的相等测试找出所有Fun4All。

```
FROM Customers
WHERE cust_name = 'Fun4All';
```

-- 组合上面的两条语句

```
SELECT cust_name, cust_contact, cust_email
FROM Customers
WHERE cust_state IN ('IL','IN','MI')
UNION
SELECT cust_name, cust_contact, cust_email
FROM Customers
WHERE cust_name = 'Fun4All';
```

使用规则

1. UNION 必须由两条或两条以上的 SELECT 语句组成，语句之间用关键字 UNION 分隔。

2. UNION 中的每个查询必须包含相同的列、表达式或聚集函数。
3. 列数据类型必须兼容：类型不必完全相同，但必须是 DBMS 可以隐含转换的类型（例如，不同的数值类型或不同的日期类型）。

包含或者取消重复行



使用 union 时，重复的行会被自动取消。

union all：让 DBMS 不再自动取消重复的行。

UNION 几乎总是完成与多个 WHERE 条件相同的工作。UNION ALL 为 UNION 的一种形式，它完成 WHERE 子句完成不了的工作。如果确实需要每个条件的匹配行全部出现（包括重复行），就必须使用 UNION ALL，而不是 WHERE。

排序



在用 UNION 组合查询时，只能使用一条 ORDER BY 子句，它必须位于最后一条 SELECT 语句之后，并且不允许使用多条 ORDER BY 子句。

```
SELECT cust_name, cust_contact, cust_email
FROM Customers
WHERE cust_state IN ('IL','IN','MI')
UNION
SELECT cust_name, cust_contact, cust_email
FROM Customers
WHERE cust_name = 'Fun4All'
ORDER BY cust_name, cust_contact;
```

14. 插入数据 - INSERT

插入完整的行



存储到表中每一列的数据在 VALUES 子句中给出，必须给每一列提供一个值。如果某列没有值，则使用 NULL 值。各列必须以它们在表定义中出现的次序填充。

```
-- 将一个新顾客插入到 Customers 表中。
INSERT INTO Customers
VALUES('1000000006',
'Toy Land',
'123 Any Street',
'New York',
'NY',
'11111',
'USA',
NULL,
NULL);
```



虽然这种语法很简单，但并不安全。高度依赖于表中列的定义次序，还依赖于其容易获得的次序信息。即使可以得到这种次序信息，也不能保证各列在下一次表结构变动后保持完全相同的次序。

插入行的一部分

```
-- 明确给出表的列名
INSERT INTO Customers(cust_id,
    cust_name,
    cust_address,
    cust_city,
    cust_state,
    cust_zip,
    cust_country)
VALUES('1000000006',
    'Toy Land',
    '123 Any Street',
```

```
'New York',  
'NY',  
'11111',  
'USA');
```

插入某些查询的结果 - INSERT SELECT

```
-- 把另一表中的顾客列合并到 Customers 表中  
INSERT INTO Customers(cust_id, -- 从 CustNew 中 将 所 有 数  
数据 导 入  
Customers  
    cust_contact,  
    cust_email,  
    cust_name,  
    cust_address,  
    cust_city,  
    cust_state,  
    cust_zip,  
    cust_country)  
SELECT cust_id, --从 CustNew 检索出要插入的值  
    cust_contact,  
    cust_email,  
    cust_name,  
    cust_address,  
    cust_city,  
    cust_state,  
    cust_zip,  
    cust_country  
FROM CustNew;
```



DBMS 不关心 SELECT 返回的列名。它使用的是列的位置，因此 SELECT 中的第一列（不管其列名）将用来填充表列中指定的第一列，第二列将用来填充表列中指定的第二列，如此等等。

INSERT SELECT 中 SELECT 语句可以包含 WHERE 子句，以过滤插入的数据。

INSERT 通常只插入一行。要插入多行，必须执行多个 INSERT 语句。

复制表 - SELECT INTO

-- 创建一个名为 CustCopy 的新表，并把 Customers 表的整个内容复制到新表中。

```
SELECT *  
INTO CustCopy  
FROM Customers;
```

-- MariaDB、MySQL、Oracle、PostgreSQL 和 SQLite 使用的语法稍有不同：

```
create table custcopy as  
select * from customers;
```



在使用 SELECT INTO 时：

1. 任何 SELECT 选项和子句都可以使用，包括 WHERE 和 GROUP BY；
2. 可利用联结从多个表插入数据；
3. 不管从多少个表中检索数据，数据都只能插入到一个表中。

15. 更新与删除

更新 - UPDATE



两种使用方式：

1. 更新表中的特定行；
2. 更新表中的所有行。

UPDATE 语句组成部分：

1. 要更新的表；
2. 列名和它们的新值；
3. 确定要更新哪些行的过滤条件。

```
-- 客户 1000000005 现在有了电子邮件地址、  
UPDATE Customers  
SET cust_email = 'kim@thetoystore.com' --SET 命令用来将新值赋  
给被更新的列  
WHERE cust_id = '1000000005';
```

```
-- 更新多个列  
-- 更新顾客 1000000006的 cust_contact 和 cust_email 列  
UPDATE Customers  
SET cust_contact = 'Sam Roberts',  
    cust_email = 'sam@toyland.com'  
WHERE cust_id = '1000000006';
```

```
-- 要删除某个列的值，可设置它为 NULL  
UPDATE Customers  
SET cust_email = NULL  
WHERE cust_id = '1000000005';
```

删除数据 - DELETE



两种方式：

1. 从表中删除特定的行
2. 从表中删除所有行

```
-- 从 Customers 表中删除一行
-- DELETE FROM 要求指定从中删除数据的表名，WHERE 子句过滤要删除的行。
DELETE FROM Customers
WHERE cust_id = '1000000006'
```



DELETE 不需要列名或通配符。DELETE 删除整行而不是删除列。要删除指定的列，请使用 UPDATE 语句。

更新和删除的指导原则

1. 除非确实打算更新和删除每一行，否则绝对不要使用不带 WHERE 子句的 UPDATE 或 DELETE 语句。（如果执行 UPDATE/DELETE 而不带 WHERE 子句，则表中每一行都将用新值更新。）
2. 保证每个表都有主键。
3. 在 UPDATE 或 DELETE 语句使用 WHERE 子句前，应该先用 SELECT 进行测试，保证它过滤的是正确的记录，以防编写的 WHERE 子句不正确。
4. 使用强制实施引用完整性的数据库，这样 DBMS 将不允许删除其数据与其他表相关联的行。
5. 有的 DBMS 允许数据库管理员施加约束，防止执行不带 WHERE 子句的 UPDATE 或 DELETE 语句。

16. 创建与操纵表

创建表 - CREATE TABLE

```
CREATE TABLE Products
(
  prod_id CHAR(10) NOT NULL,
  vend_id CHAR(10) NOT NULL,
  prod_name CHAR(254) NOT NULL,
  prod_price DECIMAL(8,2) NOT NULL,
  prod_desc VARCHAR(1000) NULL
);
```

NULL 值



NULL 值就是没有值或缺值。

允许 NULL 值的列也允许在插入行时不给出该列的值。不允许 NULL 值的列不接受没有列值的行，换句话说，在插入或更新行时，该列必须有值。

s

```
CREATE TABLE Vendors
(
  vend_id CHAR(10) NOT NULL,
  vend_name CHAR(50) NOT NULL,
  vend_address CHAR(50) ,
  vend_city CHAR(50) ,
  vend_state CHAR(5) ,
  vend_zip CHAR(10) ,
  vend_country CHAR(50)
);
```

指定默认值



SQL 允许指定默认值，在插入行时如果不给出值，DBMS 将自动采用默认值。默认值在 CREATE TABLE 语句的列定义中用关键字 DEFAULT 指定。

```
CREATE TABLE OrderItems
(
order_num INTEGER NOT NULL,
order_item INTEGER NOT NULL,
prod_id CHAR(10) NOT NULL,
quantity INTEGER NOT NULL DEFAULT 1, -- 指示 DBMS，如果不给出
数量则使用数量 1。
item_price DECIMAL(8,2) NOT NULL
);
```

更新表 - ALTER TABLE



1. 所有的 DBMS 都允许给现有的表增加列，不过对所增加列的数据类型（以及 NULL 和 DEFAULT 的使用）有所限制。
2. 许多 DBMS 不允许删除或更改表中的列。
3. 多数 DBMS 允许重新命名表中的列。
4. 许多 DBMS 限制对已经填有数据的列进行更改，对未填有数据的列几乎没有限制。
5. 在 ALTER TABLE 之后给出要更改的表名。
6. 列出要做哪些更改。

```
-- 给 Vendors 表增加一个名为 vend_phone 的列，其数据类型为 CHAR。
ALTER TABLE Vendors
ADD vend_phone CHAR(20);
```

删除表 - DROP TABLE

```
-- 删除 CustCopy 表
DROP TABLE CustCopy;
```

重命名表 - 不同DBMS标准不同

17. 视图



视图的常见应用：

1. 重用 SQL 语句。
2. 简化复杂的 SQL 操作。
3. 使用表的一部分而不是整个表。
4. 保护数据。
5. 更改数据格式和表示。



视图的规则和限制：

1. 与表一样，视图必须唯一命名（不能给视图取与别的视图或表相同的名字）。
2. 对于可以创建的视图数目没有限制。
3. 创建视图，必须具有足够的访问权限。这些权限通常由数据库管理人员授予。
4. 视图可以嵌套，即可以利用从其他视图中检索数据的查询来构造视图。（所允许的嵌套层数在不同的 DBMS 中有所不同）
5. 许多 DBMS 禁止在视图查询中使用 ORDER BY 子句。
6. 有些 DBMS 要求对返回的所有列进行命名，如果列是计算字段，则需要使用别名。
7. 视图不能索引，也不能有关联的触发器或默认值。
8. 有些 DBMS 把视图作为只读的查询，这表示可以从视图检索数据，但不能将数据写回底层表。
9. 有些 DBMS 允许创建这样的视图，它不能进行导致行不再属于视图的插入或更新。

创建视图 - CREATE VIEW

利用视图简化复杂的联结

-- 创建一个名为 ProductCustomers 的视图，它联结三个表，返回已订购了任意产品的所有顾客的列表。

```
CREATE VIEW ProductCustomers AS
SELECT cust_name, cust_contact, prod_id
FROM Customers, Orders, OrderItems
WHERE Customers.cust_id = Orders.cust_id
AND OrderItems.order_num = Orders.order_num;
```

-- 检索订购了产品 RGAN01 的顾客

```
SELECT cust_name, cust_contact
FROM ProductCustomers
WHERE prod_id = 'RGAN01';
```

用视图重新格式化检索出的数据

-- 在单个组合计算列中返回供应商名和位置

```
SELECT RTRIM(vend_name) + ' ('
+ RTRIM(vend_country) + ')'
      AS vend_title
FROM Vendors
ORDER BY vend_name;
```

```
vend_title
-----
-----
-----
Bear Emporium (USA)
Bears R Us (USA)
Doll House Inc. (USA)
Fun and Games (England)
Furball Inc. (USA)
Jouets et ours (France)
```



上述语句中的 “+” 也可写作 “||”

-- 把此语句转换为视图，不必在每次需要时执行这种拼接

```
CREATE VIEW VendorLocations AS
SELECT RTRIM(vend_name) + ' (' + RTRIM(vend_country) + ')'
      AS vend_title
FROM Vendors;
```

用视图过滤不想要的数据库

```
-- 定义CustomerEMailList 视图, 过滤没有电子邮件地址的顾客
CREATE VIEW CustomerEMailList AS
SELECT cust_id, cust_name, cust_email
FROM Customers
WHERE cust_email IS NOT NULL; -- WHERE 子句过滤了 cust_email
列中具有 NULL 值的那些行
```

视图与计算字段

```
-- 检索某个订单中的物品, 计算每种物品的总价格 (非视图)
SELECT prod_id,
       quantity,
       item_price,
       quantity*item_price AS expanded_price
FROM OrderItems
WHERE order_num = 20008;
```

```
-- 将上述语句其转换为一个视图
CREATE VIEW OrderItemsExpanded AS
SELECT order_num,
       prod_id,
       quantity,
       item_price,
       quantity*item_price AS
expanded_price
FROM OrderItems;
-- 检索订单 20008 的详细内容
SELECT *
FROM OrderItemsExpanded
WHERE order_num = 20008;
```

order_num	prod_id	quantity	item_price	expanded_price
20008	RGAN01	5	4.99	24.95
20008	BR03	5	11.99	59.95
20008	BNBG01	10	3.49	34.90
20008	BNBG02	10	3.49	34.90
20008	BNBG03	10	3.49	34.90

18. 存储过程



单独编写每条 SQL 语句，并根据结果有条件地执行其他语句。

简单来说，存储过程就是为以后使用而保存的一条或多条 SQL 语句。可将其视为批文件，虽然它们的作用不仅限于批处理

存储过程的优势：

1. 通过把处理封装在一个易用的单元中，可以简化复杂的操作
2. 由于不要求反复建立一系列处理步骤，因而保证了数据的一致性。
3. 简化对变动的管理。
4. 因为存储过程通常以编译过的形式存储，所以 DBMS 处理命令所需的工作量少，提高了性能。
5. 存在一些只能用在单个请求中的 SQL 元素和特性，存储过程可以使用它们来编写功能更强更灵活的代码。

存储过程的缺陷：

1. 不同 DBMS 中的存储过程语法有所不同。
2. 一般来说，编写存储过程比编写基本 SQL 语句复杂

执行存储过程 - EXECUTE

```
-- 执行一个名为 AddNewProduct 的存储过程，将一个新产品添加到Products 表中。
-- AddNewProduct 有四个参数，分别是：供应商 ID (Vendors 表的主键)、产品名、价格和描述。
-- 这 4 个参数匹配存储过程中 4 个预期变量（定义为存储过程自身的组成部分）。
-- 此存储过程将新行添加到 Products 表，并将传入的属性赋给相应的列。
EXECUTE AddNewProduct( 'JTS01',
    'Stuffed Eiffel Tower',
    6.49,
    'Plush stuffed toy with the text La
    ↳Tour Eiffel in red white and blue' );
```



存储过程所完成的工作：

1. 验证传递的数据，保证所有 4 个参数都有值；
2. 生成用作主键的唯一 ID；
3. 将新产品插入 Products 表，在合适的列中存储生成的主键和传递的数据。

创建存储过程

- 对邮件发送清单中具有邮件地址的顾客进行计数
- Oracle，支持IN（传递值给存储过程）、OUT（从存储过程返回值）、INOUT（既传递值给存储过程也从存储过程传回值）类型的参数。
- 这个存储过程有一个名为 ListCount 的参数。
- 此参数从存储过程返回一个值而不是传递一个值给存储过程，关键字 OUT 用来指示这种行为。
- 过程的代码括在 BEGIN 和 END 语句中
- 这里执行一条简单的 SELECT 语句，它检索具有邮件地址的顾客。然后用检索出的行数设置 ListCount（要传递的输出参数）。

```
CREATE PROCEDURE MailingListCount (  
    ListCount OUT INTEGER  
)  
IS  
    v_rows INTEGER;  
BEGIN  
    SELECT COUNT(*) INTO v_rows  
    FROM Customers  
    WHERE NOT cust_email IS NULL;  
    ListCount := v_rows;  
END;
```

- 以下为调用Oracle例子：
 - 这段代码声明了一个变量来保存存储过程返回的任何值，然后执行存储过程，再使用 SELECT 语句显示返回的值。
- ```
var ReturnValue NUMBER
EXEC MailingListCount(:ReturnValue);
SELECT ReturnValue;
```



## 19. 管理事务处理



使用事务处理 (transaction processing)，通过确保成批的 SQL 操作要么完全执行，要么完全不执行，来维护数据库的完整性。

一些相关术语：

1. 事务 (transaction) 指一组 SQL 语句。
2. 回退 (rollback) 指撤销指定 SQL 语句的过程；
3. 提交 (commit) 指将未存储的 SQL 语句结果写入数据库表；
4. 保留点 (savepoint) 指事务处理中设置的临时占位符 (placeholder)，可以对它发布回退 (与回退整个事务处理不同)。

### 控制事务处理



管理事务的关键在于将 SQL 语句组分解为逻辑块，并明确规定数据何时应该回退，何时不应该回退。

有的 DBMS 要求明确标识事务处理块的开始和结束，

```
-- 在 SQL Server 中，标识如下：
-- BEGIN TRANSACTION 和 COMMIT TRANSACTION 语句之间的
SQL 必须完全执行或者完全不执行
BEGIN TRANSACTION
...
COMMIT TRANSACTION
-- Maria DB 和 Mysql中：
START TRANSACTION
...
-- oracle：
SET TRANSACTION
...
-- PostgreSQL 使用 ANSI SQL 语法：
BEGIN
...
```

## 使用ROLLBACK

```
-- 执行 DELETE 操作，然后用 ROLLBACK 语句撤销。
DELETE FROM Orders;
ROLLBACK;
```

## 使用COMMIT



一般的 SQL 语句都是针对数据库表直接执行和编写的。这就是所谓的隐式提交 (implicit commit)，即提交 (写或保存) 操作是自动进行的。但是在事务处理块中，提交不会隐式进行。

```
-- 从系统中完全删除订单 12345。
-- 涉及更新两个数据库表 Orders 和 OrderItems，所以使用事务处理块来
保证订单不被部分删除。最后的 COMMIT 语句仅在不出错时写出更改。
-- 如果第一条 DELETE 起作用，但第二条失败，则 DELETE 不会提交。
-- SQL server
BEGIN TRANSACTION
DELETE OrderItems WHERE order_num = 12345
DELETE Orders WHERE order_num = 12345
COMMIT TRANSACTION
-- Oracle
SET TRANSACTION
DELETE OrderItems WHERE order_num = 12345;
DELETE Orders WHERE order_num = 12345;
COMMIT;
```

## 使用保留点



要支持回退部分事务，必须在事务处理块中的合适位置放置占位符。这样，如果需要回退，可以回退到某个占位符。在 SQL 中，这些占位符称为保留点。  
每个保留点都要取能够标识它的唯一名字，以便在回退时，DBMS 知道回退到何处。

```
-- MariaDB、MySQL 和 Oracle
SAVEPOINT deletel;
-- SQL Server
SAVE TRANSACTION deletel;
```

```
-- 要回退到本例给出的保留点
-- SQL server
ROLLBACK TRANSACTION deletel;
-- MariaDB、MySQL 和 Oracle
ROLLBACK TO deletel;
```

```
-- 一个完整的 SQL Server 例子：
-- 事务处理块中包含了 4 条 INSERT 语句。
BEGIN TRANSACTION
INSERT INTO Customers(cust_id, cust_name)
VALUES('1000000010', 'Toys Emporium');
SAVE TRANSACTION StartOrder; -- 定义了一个保留点
INSERT INTO Orders(order_num, order_date, cust_id)
VALUES(20100, '2001/12/1', '1000000010');
IF @@ERROR <> 0 ROLLBACK TRANSACTION StartOrder;
INSERT INTO OrderItems(order_num, order_item, prod_id, quan
tity,
 item_price)
VALUES(20100, 1, 'BR01', 100, 5.49);
IF @@ERROR <> 0 ROLLBACK TRANSACTION StartOrder;
INSERT INTO OrderItems(order_num, order_item, prod_id, quan
tity,
 item_price)
VALUES(20100, 2, 'BR03', 100, 10.99);
IF @@ERROR <> 0 ROLLBACK TRANSACTION StartOrder;
COMMIT TRANSACTION
```

## 20. 游标



简单地使用 SELECT 语句，没有办法得到第一行、下一行或前 10 行。需要在检索出来的行中前进或后退一行或多行，这就是游标的用途所在。游标（cursor）是一个存储在 DBMS 服务器上的数据库查询，它不是一条 SELECT 语句，而是被该语句检索出来的结果集。在存储了游标之后，应用程序可以根据需要滚动或浏览其中的数据。



常见的一些选项和特性：

1. 能够标记游标为只读，使数据能读取，但不能更新和删除。
2. 能控制可以执行的定向操作（向前、向后、第一、最后、绝对位置、相对位置等）
3. 能标记某些列为可编辑的，某些列为不可编辑的。
4. 规定范围，使游标对创建它的特定请求（如存储过程）或对所有请求可访问。
5. 指示 DBMS 对检索出的数据（而不是指出表中活动数据）进行复制，使数据在游标打开和访问期间不变化。

## 使用游标



步骤：

1. 在使用游标前，必须声明（定义）它。这个过程实际上没有检索数据，它只是定义要使用的 SELECT 语句和游标选项。
2. 一旦声明，就必须打开游标以供使用。这个过程用前面定义的 SELECT 语句把数据实际检索出来。
3. 对于填有数据的游标，根据需要取出（检索）各行。
4. 在结束游标使用时，必须关闭游标，可能的话，释放游标（有赖于具体的 DBMS）。

## 创建游标 - DECLARE



DECLARE 命名游标，并定义相应的 SELECT 语句，根据需要带 WHERE 和其他子句。

```
-- 创建一个游标来检索没有电子邮件地址的所有顾客
-- DECLARE 语句用来定义和命名游标，这里为CustCursor。
-- DB2、MariaDB、MySQL 和 SQL Server
DECLARE CustCursor CURSOR
FOR
SELECT * FROM Customers
WHERE cust_email IS NULL
-- Oracle 和 PostgreSQL
DECLARE CURSOR CustCursor
IS
SELECT * FROM Customers
WHERE cust_email IS NULL
```

## 使用游标 - OPEN CURSOR

```
OPEN CURSOR CustCursor
-- 现在可以用 FETCH 语句访问游标数据了。
-- FETCH 指出要检索哪些行，从何处检索它们以及将它们放于何处（如变量名）。
-- Oracle
DECLARE TYPE CustCursor IS REF CURSOR
RETURN Customers%ROWTYPE;
DECLARE CustRecord Customers%ROWTYPE
BEGIN
OPEN CustCursor;
 FETCH CustCursor INTO CustRecord; -- FETCH 用来检索当前行
 (自动从第一行开始)，放到声明的变量 CustRecord 中。
 CLOSE CustCursor; -- 对于检索出来的数据不做任何处理。
END;
```

```
-- 从第一行到最后一行，对检索出来的数据进行循环
-- Oracle
```

```

DECLARE TYPE CustCursor IS REF CURSOR
 RETURN Customers%ROWTYPE;
DECLARE CustRecord Customers%ROWTYPE
BEGIN
 OPEN CustCursor;
 LOOP
 FETCH CustCursor INTO CustRecord; -- FETCH 位于 LOOP 内
 EXIT WHEN CustCursor%NOTFOUND; -- 在取不出更多的行时终止处
理
 ...
 END LOOP;
 CLOSE CustCursor;
END;

```

```

--
DECLARE @cust_id CHAR(10),
 @cust_name CHAR(50),
 @cust_address CHAR(50),
 @cust_city CHAR(50),
 @cust_state CHAR(5),
 @cust_zip CHAR(10),
 @cust_country CHAR(50),
 @cust_contact CHAR(50),
 @cust_email CHAR(255)
OPEN CustCursor
FETCH NEXT FROM CustCursor -- 为每个检索出的列声明一个变量，FETC
H 语句检索一行并保存值到这些变量中。
 INTO @cust_id, @cust_name, @cust_address,
 @cust_city, @cust_state, @cust_zip,
 @cust_country, @cust_contact, @cust_email
WHILE @@FETCH_STATUS = 0 -- 在取不出更多的行时终止处理。
BEGIN
 FETCH NEXT FROM CustCursor
 INTO @cust_id, @cust_name, @cust_address,
 @cust_city, @cust_state, @cust_zip,
 @cust_country, @cust_contact, @cust_email

```

```
END
CLOSE CustCursor
```

## 关闭游标

```
-- DB2、Oracle 和PostgreSQL
CLOSE CustCursor
-- Microsoft SQL Server
CLOSE CustCursor
DEALLOCATE CURSOR CustCursor
```

# 21. 高级SQL特性

## 约束

### 主键



主键是一种特殊的约束，用来保证一列（或一组列）中的值是唯一的，而且永不改动。

表中的一列（或多个列）的值唯一标识表中的每一行。这方便了直接或交互地处理表中

的行。

没有主键，要安全地 UPDATE 或 DELETE 特定行而不影响其他行会非常困难。

使用主键的条件：

1. 任意两行的主键值都不相同。
2. 包含主键值的列从不修改或更新。
3. 每行都具有一个主键值
4. 主键值不能重用。如果从表中删除某一行，其主键值不分配给新行。

```
CREATE TABLE Vendors
(
vend_id CHAR(10) NOT NULL PRIMARY KEY, -- 给表的 vend_id 列
```

定义添加关键字 PRIMARY KEY，使其成为主键。

```
vend_name CHAR(50) NOT NULL,
vend_address CHAR(50) NULL,
vend_city CHAR(50) NULL,
vend_state CHAR(5) NULL,
vend_zip CHAR(10) NULL,
vend_country CHAR(50) NULL
);
```

```
-- 使用 CONSTRAINT 语法
ALTER TABLE Vendors
ADD CONSTRAINT PRIMARY KEY (vend_id);
```

## 外键



外键是表中的一列，其值必须列在另一表的主键中。外键是保证引用完整性的极其重要部分。

例：Orders 表将录入到系统的每个订单作为一行包含其中。顾客信息存储在

Customers 表中。Orders 表中的订单通过顾客 ID 与 Customers 表中的特定行相关联。顾客 ID 为 Customers 表的主键，每个顾客都有唯一的ID。订单号为 Orders 表的主键，每个订单都有唯一的订单号。Orders 表中顾客 ID 列的值不一定是唯一的。如果某个顾客有多个订单，则有多个行具有相同的顾客 ID（虽然每个订单都有不同的订单号）。同时，Orders 表中顾客 ID 列的合法值为 Customers 表中顾客的 ID。

```
-- 在 Orders 的顾客 ID 列上定义了一个外键，因此该列只能接受 Customers 表的主键值。
```

```
CREATE TABLE Orders
(
 order_num INTEGER NOT NULL PRIMARY KEY,
 order_date DATETIME NOT NULL,
 cust_id CHAR(10) NOT NULL REFERENCES -- cust_id 中的任何值都必须是 Customers 表的 cust_id 中的值。
 Customers(cust_id)
```



```
);
-- 在 ALTER TABLE 语句中用 CONSTRAINT 语法来完成:
ALTER TABLE Orders
ADD CONSTRAINT
FOREIGN KEY (cust_id) REFERENCES Customers (cust_id)
```

## 唯一约束



唯一约束的语法类似于其他约束的语法。唯一约束既可以用 `UNIQUE` 关键字在表定义中定义，也可以用单独的 `CONSTRAINT` 定义。

## 检查约束



用来保证一行（或一组行）中的数据满足一组指定的条件。  
常见用途：

1. 检查最小或最大值。
2. 指定范围。
3. 只允许特定值。

```
-- 对 OrderItems 表施加了检查约束，它保证所有物品的数量大于 0。
CREATE TABLE OrderItems
(
order_num INTEGER NOT NULL,
order_item INTEGER NOT NULL,
prod_id CHAR(10) NOT NULL,
quantity INTEGER NOT NULL CHECK (quantity > 0),
item_price MONEY NOT NULL
);
```

```
-- 检查名为gender 的列只包含 M 或 F
-- ALTER TABLE 语句:
ADD CONSTRAINT CHECK (gender LIKE '[MF]')
```

# 索引



索引用来排序数据以加快搜索和排序操作的速度。  
在一个或多个列上定义索引，使 DBMS 保存其内容的一个排过序的列表。  
在定义了索引后，DBMS 以使用书的索引类似的方法使用它。DBMS 搜索排过序的索引，找出匹配的位置，然后检索这些行。

-- 在 Products 表的产品名列上创建一个简单的索引。  
-- 这里的索引名 prod\_name\_ind 在关键字 CREATE INDEX 之后定义。  
-- ON 用来指定被索引的表，而索引中包含的列（此例中仅有一列）在表名后的圆括号中给出。

```
CREATE INDEX prod_name_ind
ON Products (prod_name);
```



索引必须唯一命名。

## 触发器



触发器是特殊的存储过程，它在特定的数据库活动发生时自动执行。触发器可以与特定表上的 INSERT、UPDATE 和 DELETE 操作（或组合）相关联。

触发器内代码的数据访问权：

1. INSERT 操作中的所有新数据；
2. UPDATE 操作中的所有新数据和旧数据；
3. DELETE 操作中删除的数据。

常见用途：

1. 保证数据一致。
2. 基于某个表的变动在其他表上执行活动。
3. 进行额外的验证并根据需要回退数据。
4. 计算计算列的值或更新时间戳。

```
-- 对所有 INSERT 和 UPDATE 操作，将 Customers 表中的 cust_state
列转换为大写。
-- SQL Server
CREATE TRIGGER customer_state
ON Customers
FOR INSERT, UPDATE
AS
UPDATE Customers
SET cust_state = Upper(cust_state)
WHERE Customers.cust_id = inserted.cust_id;
-- Oracle 和 PostgreSQL
CREATE TRIGGER customer_state
AFTER INSERT OR UPDATE
FOR EACH ROW
BEGIN
UPDATE Customers
SET cust_state = Upper(cust_state)
WHERE Customers.cust_id = :OLD.cust_id
END;
```

## 22. 语法速记

### ALTER TABLE - 更新已存在表的结构

```
ALTER TABLE tablename
(
 ADD|DROP column datatype [NULL|NOT NULL] [CONSTRAINTS],
 ADD|DROP column datatype [NULL|NOT NULL] [CONSTRAINTS],
 ...
);
```

### COMMIT - 将事务写入数据库

```
COMMIT [TRANSACTION];
```

### CREATE INDEX - 在一个或多个列上创建索引

```
CREATE INDEX indexname
ON tablename (column, ...);
```

### CREATE PROCEDURE - 创建存储过程

```
CREATE PROCEDURE procedurename [parameters] [options]
AS
SQL statement;
```

### CREATE TABLE - 创建新数据库表

```
CREATE TABLE tablename
(
 column datatype [NULL|NOT NULL] [CONSTRAINTS],
 column datatype [NULL|NOT NULL] [CONSTRAINTS],
 ...
);
```

## CREATE VIEW - 创建一个或多个表上的新视图

```
CREATE VIEW viewname AS
SELECT columns, ...
FROM tables, ...
[WHERE ...]
[GROUP BY ...]
[HAVING ...];
```

## DELETE - 从表中删除一行或多行

```
DELETE FROM tablename
[WHERE ...];
```

## DROP - 永久地删除数据库对象（表、视图、索引等）

```
DROP INDEX|PROCEDURE|TABLE|VIEW
indexname|procedurename|tablename|viewname;
```

## INSERT - 为表添加一行

```
INSERT INTO tablename [(columns, ...)]
VALUES(values, ...);
```

## INSERT SELECT - 将 SELECT 的结果插入到一个表

```
INSERT INTO tablename [(columns, ...)]
SELECT columns, ... FROM tablename, ...
[WHERE ...];
```

## ROLLBACK - 撤销一个事务块

```
ROLLBACK [TO savepointname];
```

## SELECT - 从一个或多个表（视图）中检索数据

```
SELECT columnname, ...
FROM tablename, ...
[WHERE ...]
[UNION ...]
[GROUP BY ...]
[HAVING ...]
[ORDER BY ...];
```

## UPDATE - 更新表中的一行或多行

```
UPDATE tablename
SET columnname = value, ...
[WHERE ...];
```